

Implement a Creational Pattern: Factory

Software Design & Programming

Object-Oriented Programming II

Joseph DiBiasi

University of Denver College of Professional Studies

4/25/2026

Faculty: Nathan Braun, MBA

Director: Cathie Wilson, MS

Dean: Bobbie Kite, MLS

Introduction

Last week saw the university Parking System lay down a blueprint for how it wanted to build its configurable parking strategies; this week we have implemented the creation of those blueprints into concrete parking strategies usable by our parking lots. This work represents the culmination of our initial goal of creating a flexible parking strategy builder. We want to allow our parking lot management team the maximum amount of freedom in how they build these strategies so that we can optimize the usage of our parking lots. This was done by building a core of generic parking strategies that could take in parameters to adjust them. These strategies could then be combined to build more complex strategies for each parking lot.

Requirements

The requirements for this work are closely linked with those in our previous development cycle. After all, what good is designing our parking strategies if we have no way of implementing them? This meant we were still working with a core requirement of needing to design a flexible strategy to calculate different rates for each parking lot operated by our parking office. This week added the additional requirement of actually being able to create those classes and assign them to parking lots. This build process required the same flexibility concerns that accompanied our strategy design phase; a factory pattern was therefore desired.

High-Level Design

The creation of our parking strategies is centered around our parking strategy factory. This factory has a single focus; the creation of parking strategies. The creation process is designed to be dynamic at runtime, this is accomplished through the use of two input parameters, strategy type and strategy configuration. Strategy type is styled as Enums, providing a controlled method of deciding what strategy is built by the factory while still providing flexibility to add or remove new strategies due to the switch statement. Strategy configuration is how each individual parking strategy can be defined in a multitude of ways, containing ways to modify a strategy's rate as well as define what triggers it, such as specific days of the week. To keep this configuration flexible as the number of strategies grow, only the rate modifier is a required check. Everything else is only validated on the basis of need for the specific strategy that needs it. This lets us easily add or remove strategies to the factory without disrupting other classes. The factory is then utilized by the transaction manager to assign strategies to parking lots.

Implementation

The parking charge strategy factory is focused solely on the creation of our parking strategies. There is minimal error handling, it is limited only to ensuring that the required parameters for strategy creation are present. Strategy creation is decided at runtime by the strategy type Enum. Strategy configuration is a generic class passed into each strategy the factory constructs. The configuration class utilizes a builder pattern to allow customizable object creation, meeting the needs of each strategy.

The actual process for invoking the factory pattern workflow belongs to the transaction manager class. A new method to assign strategies dynamically was added; this method allows the creation and removal of parking lot strategies. Our overall strategy implementation design was centered around small building blocks being combined into complex strategies, this is reflected in being able to assign multiple parking strategies to a single parking lot via a list of strategy data transfer objects (DTO). An empty list results in a parking lot being assigned no strategies. The strategy DTO contains a pairing of strategy type and strategy configuration objects. This ensures that when the factory is called to build a strategy, it is passed a matching type and configuration. While reconfiguring the hardcoded strategy values into configurable parameters, it was decided to standardize the rate modifier calculations for each strategy. This enhanced readability greatly from a development perspective as different calculations were not needed for each strategy, only different parameters.

The creation of the new factory method as well as the rework of some existing methods required additional test cases and classes to be added to the application to achieve full coverage. The use of Enums in the main factory class caused issues covering each step of the switch statement as it was not possible to reach the default. An attempt was made to cover this case through mocking an Enum, but eventually it was decided to leave the default in as a defensive measure while leaving it uncovered as it is unreachable code under normal means.

Summary

The completion of our flexible parking strategy workflow felt like it was a highlight of following good object-oriented practices from the start. Our strong pattern focused design in our previous

development cycle was further boosted by a mid-cycle rework of our address, customer, and parking charge classes into utilizing the builder pattern. This strong foundation allowed the easy creation of our factory class. Additional changes were made to the application to improve consistency of calculations and further embrace the similarities in functionality. The end result was a strategy builder that could create far more parking strategies than the initial core four strategies suggested. The number of core strategies could also be easily improved simply by following the existing pattern for new strategies with their own unique twist then adding them to the factory for assignment. Taking extra care in our design phase to be aligned with future developments provided a benefit we are sure not to forget.